

BERT

So before the BERT, language models generally can only:

- **Left-to-right models like GPT**

For example:

The cat sat on the _____ but the prediction can only see The cat sat on the So the model cannot see the future words. `` `

- **Right-to-left models**

- **For example:**

_____ sat on the mat but the prediction can only see sat on the mat So the model cannot see the future words.

But we have to acknowledge that many NLP tasks require understanding both the left and right context simultaneously.

For example:

- I went to the bank to deposit the money.
- I sat on the bank of the river.

Both sentences have the word "bank", but to understand what this word means, we need to know the words before and after it to understand the context. And this is when the BERT comes into play.

BERT use the **Bidirectional Context** by using the transformer encoder.

So it is used to predict the missing/masked words.

For example:

- The cat sat on the mat.
- The cat [*Mask*] on the mat.

So the model need to predict the word "sat" and when predicting the [*Mask*] the model can see "The cat" and the "on the mat"

Which is why it is called bidirectional context language model.

and the reason that it does not use the decoder is simple as the decoder is used for generation text, but generation tasks are not what the BERT trying to improve, but it need to understand the context, the relationship in the sentence.

Input Representation

Unlike the original transformer, BERT combines three embeddings together.

So for every token:

Final embedding = Token Embedding + Position Embedding + Segment Embedding

Token Embedding

It convert words into vectors

```
I  
↓  
[0.4, 0.1, ...]
```

Position Embedding

To our knowledge, attention has notion of order. So the BERT adds the position embedding so it can learn these position embeddings during the training rather than using fixed sinusoidal ones.

```
Position 0  
↓  
vector  
  
Position 1  
↓  
vector  
  
...
```

Segment Embedding

Only used when there are two sentences

For example: if we have:

- Sentence A: I love cats.
- Sentence B: Dogs are cute.

So the input becomes

```
[CLS]  
I  
love  
cats  
[SEP]  
Dogs  
are  
cute  
[SEP]
```

Tokens from sentence A receive Segment A embeddings.

Tokens from sentence B receive Segment B embeddings.

This allows the model to distinguish which sentence each token belongs to.

Special Tokens

[*CLS*]

it is inserted in the beginning.

```
[CLS]
I
love
cats
```

And the final hidden state of [*CLS*] becomes a summary of the whole sequence and is used for sentence-level tasks like classification.

[*SEP*]

It is used to mark sentence boundaries.

```
Sentence A
[SEP]
Sentence B
[SEP]
```

[*MASK*]

It is used only during the pre-training

BERT's Training Objective

Masked Language Modeling (MLM)

MLM to predict the hidden word or missing word.

Step 1: We need to choose 15% of the tokens randomly

Step 2: replace the word according to the famous 80-10-10 rule

We suppose that "sat" was selected

Then 80%, we replace it with [*MASK*]

The cat [MASK] on the mat.

And for the 10%, replace it with random word.

The cat apple on the mat.

And for the last 10%, keep it unchanged

The cat sat on the mat.

Remember: We should not mask every word because during the fine-tuning, the model would never see the [*MASK*], because real-world inputs do not contain it. So this creates a

mismatch between pretraining and downstream tasks.

So the 80-10-10 strategy reduces the mismatch while still teaching the model to infer the missing words.

And we also should stick to the 15% rule because suppose we masked 90% of the sentence, then there would be almost no context left. Thus, learning would be inefficient.

So 15% is good:

- Enough prediction targets to learn from
- Enough visible context to infer the masked tokens.

Next Sentence Prediction (NSP)

It answers whether the sentence A actually follows sentence B or vice versa.

By doing that, it teaches the model to learn the relationships between sentences, which could be helpful for tasks like:

- Question answering
- Natural language Inference
- Sentence pair classification

Label: $[IsNext]$ = Positive

```
Sentence A
I went to school.
Sentence B
I attended my math class.
```

Label: $[NotNext]$ = Negative

```
Sentence A
I went to school.
Sentence B
Pizza tastes delicious.
```

Later research (for example, RoBERTa: A Robustly Optimized BERT Pretraining Approach) showed that NSP was not essential and removed it while still achieving better performance.

Encoder Stack

BERT Base contains:

- 12 Transformer encoder layers
- 12 attention heads
- **Hidden size:** 768
- About 110 million parameters

BERT Large contains:

- 24 encoder layers

- 16 attention heads
- Hidden size: 1024
- About 340 million parameters

Architecture

```
Input
↓
Multi-Head Self-Attention
↓
Add & LayerNorm
↓
Feed-Forward Network
↓
Add & LayerNorm
```

Finetuning

So instead of training a separate model for every NLP task

BERT follows a two-stage process:

1. **Pretrain** once on massive unlabeled text
2. **Fine-tune** on a specific task with a small task-specific output layer.

For example: finetune the model for:

- Sentiment analysis
- Question Answering: Context + Question = Prediction answer span
- Named entity recognition

Note: the same pretrained encoder is reused across many tasks.